

RUBIN SCOUT

Sprint Report

ILMT Follow-Up API Endpoint

Prepared by: Namrata Modha

Sprint Duration: May 19, 2026 | Version: 1.0

github.com/Namrata-Modha/rubin-scout

1. Sprint Overview

Sprint Name	ILMT Follow-Up API Endpoint
Sprint Date	May 19, 2026
Developer	Namrata Modha
Project	Rubin Scout v0.1
Sprint Goal	Build a single API endpoint that consolidates four manual astronomical lookups into one call, designed for direct integration with the PyLMT pipeline at ARIES/ILMT
Estimated Effort	4 hours
Status	In Development
Live URL	rubin-scout.vercel.app
GitHub	github.com/Namrata-Modha/rubin-scout

2. Background and Problem Statement

2.1 ILMT Operational Context

The International Liquid Mirror Telescope (ILMT) at Devasthal Observatory, Nainital, India is a 4-metre zenith-pointing telescope operated by ARIES. Its liquid mercury mirror cannot tilt, so it uses Time Delay Integration (TDI) drift-scanning to capture a fixed 22 arcminute strip of sky centred at declination +29° 21' 40" every night. This nightly cadence gives ILMT deep, highly standardized baseline imaging of the same sky region.

When the PyLMT pipeline detects a new transient in this strip through automated image subtraction, the science team must manually answer four questions before deciding whether to commit Devasthal Optical Telescope (DOT) spectroscopy time:

- Has ZTF already seen this object? Is it genuinely new or a known source?
- Does SIMBAD know this object? Is it a catalogued star, galaxy, or AGN?
- Did a gravitational wave or high-energy event occur near this position recently?
- Can DOT actually observe this target tonight from Devasthal?

Each question currently requires a separate manual lookup across ALeRCE, SIMBAD, GWOSC, and a separate visibility tool. This is the workflow gap Rubin Scout's ILMT Follow-Up API endpoint addresses.

2.2 Why This Matters for Multi-Messenger Science

ILMT's fixed +29° declination strip means that when a GW event from LIGO/Virgo/KAGRA overlaps this region, ILMT already has pre-trigger nightly baseline imaging without needing to re-point. This makes it uniquely positioned for optical counterpart searches, particularly for kilonovae following neutron star mergers. The Rubin Scout endpoint surfaces this GW coincidence check automatically alongside ZTF cross-survey data, reducing the time between PyLMT detection and DOT spectroscopy decision.

3. User Story

As an ILMT astronomer, I want to query a single API endpoint with my detection coordinates and MJD so that I can receive a consolidated follow-up recommendation without performing four separate manual lookups across ALeRCE, SIMBAD, GWOSC, and a visibility tool.

4. Functional Requirements

Endpoint	GET /api/ilmt/followup
Rate Limit	30 requests per minute
Required Params	ra (float, 0-360), dec (float, -90 to 90), mjd (float)
Optional Params	radius_arcsec (float, default 5.0)
Auth	None - public endpoint
Target Latency	Under 3 seconds end-to-end

4.1 ZTF Prior History (FR-ILMT-01)

Query the objects table using PostGIS ST_DWithin for all existing ZTF alerts within radius_arcsec of the given RA/Dec. For each match return: oid, classification, classification_probability, first_detection, last_detection, n_detections, a boolean indicating whether it was detected before the given MJD (pre-existing source), a boolean indicating post-MJD activity (new brightening), and the ALeRCE alert URL.

4.2 SIMBAD Association (FR-ILMT-02)

Run a SIMBAD cone search at the given coordinates within 10 arcseconds using the existing EnrichmentService._query_simbad() method. Return name, object type, and angular distance in arcseconds if a match is found. Return null if no match. Run in asyncio.to_thread to avoid blocking the event loop.

4.3 GW Coincidence Check (FR-ILMT-03)

Query the gw_events table for any event where the event_time falls within 30 days before the given MJD AND the RA/Dec falls within the event's 90% credible region, using ra_center, dec_center, and area_90_deg2 from the properties JSONB column. For each match return: superevent_id, event_time, dominant classification type, distance_mpc, and angular separation from the given coordinates.

4.4 Visibility at Devasthal (FR-ILMT-04)

Call the existing _compute_visibility() function using Devasthal coordinates from OBSERVATORY_PRESETS. Return observable (bool), max_altitude, dark_start, dark_end, and moon_separation only. Do not return the full hourly altitude array in this endpoint - keep the response compact for pipeline consumers.

4.5 Recommendation Engine (FR-ILMT-05)

Based on the four data sources above, return a single recommendation string and a plain English reason. The recommendation logic is deterministic:

Value	Trigger Conditions	Intended Action
PRIORITY_FOLLOWUP	No prior ZTF history (genuinely new) AND no known SIMBAD association (or ambiguous type) AND GW coincidence exists OR classification is KN / TDE / SNIa	<i>Commit DOT spectroscopy time immediately</i>
NEEDS_MORE_DATA	Fewer than 3 ZTF detections OR SIMBAD match is ambiguous OR new detection with no GW coincidence and uncertain classification	<i>Monitor for 1-2 more nights before spectroscopy</i>
LIKELY_KNOWN	Strong SIMBAD match to known variable star, AGN, or QSO OR more than 10 prior ZTF detections indicating a known source	<i>Deprioritize. Log and move on.</i>

5. Response Schema

The endpoint returns a single JSON object with seven top-level fields:

Field	Type	Description
<code>query</code>	<i>object</i>	Echo of input: ra, dec, mjd, radius_arcsec
<code>ztf_history</code>	<i>array</i>	All ZTF alerts within radius_arcsec. Each entry includes oid, classification, probability, first/last detection, n_detections, pre/post MJD flags, ALerCE link
<code>simbad</code>	<i>object null</i>	Nearest SIMBAD match within 10 arcsec: name, object type, distance_arcsec. Null if no match
<code>gw_coincidence</code>	<i>array</i>	GW events within 30 days of MJD overlapping the given position: superevent_id, event_time, classification type, distance_mpc, angular separation
<code>visibility_devasthal</code>	<i>object</i>	Devasthal visibility for tonight: observable (bool), max_altitude (deg), dark_start, dark_end, moon_separation (deg)
<code>recommendation</code>	<i>string</i>	One of: PRIORITY_FOLLOWUP, NEEDS_MORE_DATA, LIKELY_KNOWN
<code>recommendation_reason</code>	<i>string</i>	Plain English explanation of the recommendation

5.1 Example Request

```
GET /api/ilmt/followup?ra=185.73&dec=29.37&mjd=60812.5&radius_arcsec=5.0
```

This queries for a detection at RA 185.73°, Dec +29.37° (within ILMT's strip) detected on MJD 60812.5, searching a 5 arcsecond radius.

6. Acceptance Criteria

AC ID	Acceptance Criterion	Status	Notes
AC-ILMT-01	Endpoint returns HTTP 200 with all seven top-level fields present for a valid request	Pending	
AC-ILMT-02	ZTF history returns correct pre/post MJD boolean flags for a known object with prior detections	Pending	
AC-ILMT-03	SIMBAD query runs in <code>asyncio.to_thread</code> and returns a match for a known QSO position	Pending	
AC-ILMT-04	GW coincidence returns GW170817 when queried with NGC 4993 coordinates (RA 197.45, Dec -23.38) and MJD within 30 days of Aug 17 2017	Pending	Known reference event

AC-ILMT-05	Visibility returns observable=true for a circumpolar target at Devasthal during dark time	Pending	
AC-ILMT-06	PRIORITY_FOLLOWUP returned for a position with no ZTF history, no SIMBAD match, and a GW coincidence	Pending	
AC-ILMT-07	LIKELY_KNOWN returned for a known AGN position with 15+ prior ZTF detections and a SIMBAD QSO match	Pending	
AC-ILMT-08	NEEDS_MORE_DATA returned for a position with 2 prior ZTF detections and no GW coincidence	Pending	
AC-ILMT-09	SIMBAD and visibility run concurrently via asyncio.gather, not sequentially	Pending	Performance
AC-ILMT-10	Endpoint returns HTTP 400 for invalid RA (outside 0-360) or invalid dec (outside -90 to 90)	Pending	
AC-ILMT-11	Endpoint returns a response within 3 seconds for a cold request with no cached data	Pending	Latency
AC-ILMT-12	Rate limiter returns HTTP 429 after 30 requests per minute from the same IP	Pending	

7. Implementation Notes

7.1 Schema Changes

None required. This endpoint is entirely read-only and uses only existing tables: objects (PostGIS cone search), gw_events (GW coincidence), and the existing _compute_visibility() function which has no DB interaction.

7.2 Concurrency

SIMBAD queries and visibility computation are both synchronous and CPU-bound. Both must run via asyncio.to_thread. They are independent of each other and should run concurrently using asyncio.gather to minimize response latency. The ZTF history and GW coincidence queries are async database queries and run normally within the async endpoint.

7.3 GW Coincidence Approximation

The GW credible region is stored as area_90_deg2 in the properties JSONB column. The cross-match approximates the 90% credible region as a circle using $\text{radius} = \sqrt{\text{area} / \pi}$ degrees, capped at 15 degrees maximum. This is the same approximation used in the existing cross_match_event() function. Events with no ra_center or dec_center (poorly localized) are excluded from this check since no meaningful spatial match is possible.

7.4 Existing Code Reuse

- SIMBAD: `EnrichmentService._query_simbad(ra, dec, radius_arcsec=10.0)` - existing method, no duplication
- Visibility: `_compute_visibility(ra, dec, lat, lon, elevation, date)` - existing function, pass Devasthal coords from `OBSERVATORY_PRESETS`
- ZTF cone search: same `ST_DWithin` pattern as existing `/api/alerts/conesearch/query` endpoint
- GW query: same `select(GWEvent)` pattern as existing `cross_match_event()` method

8. Effort and Timeline

FR-ILMT-01 ZTF History	1 hour - cone search query, pre/post MJD flag logic
FR-ILMT-02 SIMBAD	30 minutes - reuse existing method, wrap in <code>asyncio.to_thread</code>
FR-ILMT-03 GW Coincidence	1 hour - spatial approximation logic, JSONB field parsing
FR-ILMT-04 Visibility	30 minutes - reuse existing function, trim response fields
FR-ILMT-05 Recommendation	30 minutes - deterministic logic tree, plain English reasons
Testing and validation	30 minutes - manual test with known coordinates
Total	4 hours (single developer, single session)

9. Impact on ARIES Outreach

The ILMT Follow-Up API endpoint changes the nature of the outreach email to Dr. Kuntal Misra and Dr. Brajesh Kumar from a dashboard demonstration to a concrete pipeline integration proposal. The email can now reference a specific endpoint that PyLMT could call programmatically after each transient detection, returning a structured JSON response with a recommendation rather than requiring a researcher to navigate four separate web interfaces.

This positions Rubin Scout not as a viewer for curious users but as a lightweight decision-support layer for a working research pipeline. That is a materially different and more compelling offer for an active science team.

Outreach Target	Dr. Kuntal Misra (kuntal@aries.res.in), Dr. Brajesh Kumar (brajesh@aries.res.in)
Institution	ARIES - Aryabhata Research Institute of Observational Sciences, Nainital, India
Email Status	Draft ready - send after endpoint is live and tested
Key Hook	<code>GET /api/ilmt/followup</code> - one call from PyLMT replaces four manual lookups

10. Deferred Items

- HEALPix-based GW sky map integration for precise credible region matching (requires healpy, not available on Windows dev environment)
- GRB/neutrino alert coincidence check - future sprint once GCN stream integration is scoped
- ILMT-specific declination strip filter ($+29^\circ \pm 0.18^\circ$) as an optional query parameter
- Caching layer for SIMBAD results to reduce repeat query latency

Appendix: Test Coordinates

Use these known positions to validate the endpoint after implementation:

GW170817 host (NGC 4993)	RA 197.45, Dec -23.38, MJD 57982.5 - should return GW coincidence with GW170817
Known QSO in ILMT strip	RA 185.73, Dec +29.37 - should return SIMBAD QSO match and prior ZTF history
Empty sky position	RA 180.0, Dec +29.36, recent MJD - should return no ZTF history, no SIMBAD, no GW
Circumpolar at Devasthal	Dec > +60 - should return observable=true during dark time